

Auditoria Frontend

Angular + Tailwind CSS

Repositorio: kennedy122112/new-siaf-rp

Fecha	29 de abril de 2026
Stack	Angular 20.3.19, Tailwind CSS 4.1.17, TypeScript 5.9.3
Revision	Arquitectura, UI Kit, Tailwind/tokens, accesibilidad, performance y build
Resultado	Base moderna, pero con deuda relevante antes de produccion

Veredicto Ejecutivo

El proyecto tiene una base tecnica prometedora: Angular moderno con componentes standalone, ChangeDetectionStrategy.OnPush, strict templates y un sistema de tokens importado desde Figma. Sin embargo, todavia no esta listo como frontend mantenible de produccion. Los principales riesgos estan en reproducibilidad del build, componentes demasiado grandes, duplicidad de UI kit, accesibilidad incompleta y uso inconsistente de Tailwind frente a los tokens semanticos.

Hallazgos Criticos

Critico - Barrel roto en shared/ui

Evidencia: src/app/shared/ui/index.ts:31 contiene `export { TextFieldComponent } ...``, sintaxis invalida. La validacion directa con TypeScript devuelve `TS1005: ',' expected``.

Impacto: Cualquier consumidor que importe desde el barrel `shared/ui`` rompera compilacion. El error puede quedar oculto si la app actual no usa ese archivo desde `main.ts``.

Recomendacion: Cambiar a `export { TextFieldComponent } from './text-field/text-field.component';`` y agregar una validacion CI que compile barrels/exportaciones publicas.

Critico - Build no reproducible bajo Node actual

Evidencia: package.json exige Node `>=20.19.0 <21`, pero el entorno auditado usa Node 24.14.1. `npm run build`` llega a esbuild y cae en deadlock nativo.

Impacto: El equipo puede tener resultados distintos entre maquinas y CI. Sin version fija, un build verde local no garantiza despliegue confiable.

Recomendacion: Fijar Node 20 LTS con `.nvmrc``, Volta o configuracion de CI. Ejecutar `npm ci && npm run build`` solo con esa version soportada.

Alto - Rutas cargadas de forma eager

Evidencia: `src/app/app.routes.ts` importa directamente todas las features: login, OTP, panel y pantallas de asiento.

Impacto: El bundle inicial incluye mas codigo del necesario para entrar al login, penalizando tiempo de carga inicial.

Recomendacion: Migrar rutas a ``loadComponent`` para lazy loading por pantalla o modulo funcional.

Arquitectura y Mantenibilidad

Alto - Pantallas monoliticas

Evidencia: `adjustment-seat-request.component.ts` tiene 1302 lineas; `adjustment-seat-form.component.ts` tiene 542; `tray-documents-view.component.ts` tiene 552.

Impacto: Dificulta ownership, revisiones, pruebas unitarias, cambios visuales y control de regresiones.

Recomendacion: Dividir en secciones de pantalla, componentes presentacionales, modelos dedicados y una facade/servicio para estado.

Alto - Duplicidad entre `shared/components` y `shared/ui`

Evidencia: Existen versiones en ambos directorios para pagination, breadcrumb, data-table, timeline, alert, snackbar y action-tracker.

Impacto: Riesgo alto de divergencia visual, APIs inconsistentes y bugs arreglados en un componente pero no en su duplicado.

Recomendacion: Definir contrato: ``shared/ui`` para atomos/base y ``shared/components`` para patrones compuestos. Eliminar duplicados o hacer wrappers explicitos.

Medio - Templates inline demasiado extensos

Evidencia: Gran parte de la app usa ``template: ```. En componentes pequenos esta bien, pero en pantallas largas vuelve pesado el archivo.

Impacto: Reduce legibilidad, dificulta linting de HTML y separacion natural entre estructura, comportamiento y estilos.

Recomendacion: Mover pantallas grandes a ``templateUrl`` y reservar templates inline para componentes pequenos.

Tailwind y Sistema de Diseno

Alto - Tokens mezclados con valores hardcodedos

Evidencia: Se observan colores y sombras directas como ``#014899``, ``rgba(...)``, ``tracking-[...]``, ``shadow-[...]`` y estilos inline.

Impacto: El sistema de diseno pierde autoridad; los cambios de marca o accesibilidad deben perseguirse en muchos archivos.

Recomendacion: Crear tokens semanticos para estados, texto, acciones, bordes y elevaciones. Restringir valores arbitrarios a excepciones documentadas.

Medio - tailwind.config.ts parcialmente redundante con Tailwind v4

Evidencia: El proyecto usa `@theme` en `styles.css` y tambien `theme.extend` en `tailwind.config.ts`.

Impacto: Puede producir confusion sobre la fuente real de verdad para tokens y utilidades.

Recomendacion: Elegir una estrategia principal: preferir `@theme` para Tailwind 4 y dejar el config solo para contenido/plugins si no es necesario extender.

Accesibilidad y UX

Alto - Modal sin focus trap ni restauracion de foco

Evidencia: El modal declara `role='dialog'` y `aria-modal`, pero no hay manejo de Escape, focus trap ni retorno del foco al trigger.

Impacto: Usuarios de teclado o lector de pantalla pueden quedar fuera de contexto o navegar contenido detras del modal.

Recomendacion: Agregar CDK Overlay/Dialog o implementar focus trap, cierre con Escape, `aria-labelledby` y `aria-describedby`.

Alto - Selects, menus y tabs con interaccion incompleta

Evidencia: TextField select y menus custom usan botones/listas visuales, pero sin patron completo de listbox/menu/keyboard navigation. TabsComponent tampoco emite seleccion.

Impacto: La UI parece interactiva, pero no escala bien para accesibilidad, formularios reales o automatizacion de pruebas.

Recomendacion: Normalizar patrones ARIA y eventos: roving tabindex, flechas, Escape, Enter/Space, roles correctos y outputs consistentes.

Medio - Inputs base sin FormControlValueAccessor

Evidencia: Checkbox, Radio y TextField no implementan una integracion completa con Reactive Forms.

Impacto: Las pantallas terminan gestionando estado manualmente y se duplica logica de validacion.

Recomendacion: Implementar FormControlValueAccessor en controles reutilizables o documentar que son puramente presentacionales.

Testing y Calidad

No se encontraron archivos `*.spec.ts`. Para un UI kit de esta escala, la ausencia de pruebas deja demasiado peso en revision manual. Priorizar pruebas de componentes base, formularios, modal, filtros, paginacion, login, OTP y una suite de accesibilidad con Playwright/axe.

Plan de Remediacion Priorizado

1	Fijar Node 20 LTS	Agregar .nvmrc/Volta y validar build en CI.
2	Corregir barrel shared/ui	Arreglar TextFieldComponent y compilar exportaciones publicas.
3	Lazy loading de rutas	Migrar a loadComponent para reducir bundle inicial.

4	Unificar UI kit	Eliminar duplicados entre shared/ui y shared/components.
5	Accesibilidad base	Modal, menus, tabs y selects con keyboard/focus completo.
6	Refactor pantallas grandes	Separar secciones, facades, modelos y componentes presentacionales.
7	Pruebas	Unitarias, accesibilidad y visual regression minima.

Comandos Ejecutados

```
npm ci --cache /private/tmp/new-siaf-rp-npm-cache --loglevel=warn
```

Resultado: dependencias instaladas, 0 vulnerabilidades.

```
npm run build
```

Resultado: falla en esbuild con deadlock bajo Node 24.14.1.

```
./node_modules/.bin/tsc -p tsconfig.app.json --noEmit
```

Resultado: OK para el arbol usado por la app.

```
./node_modules/.bin/tsc --noEmit ... src/app/shared/ui/index.ts
```

Resultado: TS1005 en el barrel de shared/ui.